

MINISTERUL EDUCAȚIEI



UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

ADAPTIVE HEADLIGHTS CONTROL FOR VEHICLES

PROIECT DE SEMESTRU

Student: **Tudor Adrian-Raul-Ștefănel**

Disciplina: **Sisteme bazate pe Cunoaștere**

2026

CUPRINS

1	Introducere.....	3
1.1	Context general.....	3
1.2	Obiective.....	3
2	Cunoașterea și analiza setului de date	5
2.1	Descrierea sursei de date	5
2.2	Mediul de implementare.....	5
2.3	Analiză statistică și structura datelor	5
3	Modelarea Sistemului.....	7
3.1	Arhitectura Modelului și Configurația de Antrenare.....	7
3.2	Testarea și Validarea	7
3.3	Detecție fază.....	8
3.4	Implementare în timp real	9
4	Concluzii	10
4.1	Rezultate obținute	10
4.2	Direcții de dezvoltare.....	13
5	Bibliografie	14

1 Introducere

1.1 Context general

În prezent, domeniul auto se schimbă foarte rapid, iar mașinile devin tot mai inteligente datorită sistemelor de asistență pentru șoferi. Cu toate acestea, condusul pe timp de noapte rămâne o problemă majoră de siguranță, vizibilitatea redusă fiind o cauză frecventă a accidentelor.

De aceea, proiectul de față, „**Adaptive Headlights Control for Vehicles**”, își propune să creeze un sistem care schimbă automat faza farurilor (lungă sau scurtă) în funcție de ce se întâmplă în trafic.

Motivația principală este legată de siguranță și confort. Știm cu toții că, noaptea, șoferii uită adesea să schimbe faza lungă cu cea scurtă atunci când vine o mașină din sens opus.

Acest lucru îi poate orbi pe ceilalți participanți la trafic sau, invers, șoferul poate rămâne cu faza scurtă pe un drum liber, unde nu vede suficient de bine. Un sistem automat elimină această grijă și reacționează mult mai rapid decât un om obosit.

Am ales să implementez acest proiect folosind **Computer Vision** (procesare de imagini) și algoritmul **YOLO** deoarece sunt tehnologii moderne și foarte performante. Din punct de vedere practic, YOLO este capabil să detecteze mașini în timp real, ceea ce este esențial pentru o mașină aflată în mișcare. Am vrut să demonstrez că se poate construi un sistem eficient care nu doar „vede” drumul, dar ia și decizii logice (să treacă pe faza scurtă) fără ca șoferul să apese vreun buton.

1.2 Obiective

Pornind de la contextul actual al siguranței rutiere și al evoluției sistemelor autonome, obiectivul fundamental al acestei lucrări este proiectarea și implementarea unui sistem software de **control adaptiv al farurilor**, bazat pe tehnici de învățare automată (Machine Learning). Scopul final este crearea unui „copilot” virtual care preia sarcina comutării fazei lungi/scurte, eliminând eroarea umană.

Pentru a atinge acest scop, mi-am propus realizarea următoarelor obiective specifice:

- **Studiul și antrenarea unui model de detecție performant**

Am urmărit familiarizarea cu arhitecturile de tip CNN și implementarea specifică a algoritmului YOLOv8.

Un punct cheie a fost utilizarea unui set de date relevant (*ML Car Headlight*), care să conțină scenarii diverse (noapte, zi, mediu urban/rural), pentru a învăța modelul să recunoască vehiculele indiferent de condițiile de iluminare.

- **Implementarea logicii de control și stabilizare**

Nu este suficient ca sistemul doar să „vadă” mașinile, ci trebuie să ia decizii corecte. Mi-am propus să dezvolt un algoritm care:

- Comută automat pe **Faza Scurtă (Low Beam)** când detectează un vehicul (pentru a nu orbi șoferul).
- Revine la **Faza Lungă (High Beam)** când drumul este liber.
- Include un mecanism de **temporizare (histerezis)** pentru a evita „pâlpâirea” farurilor (flickering) în cazul unor detecții eronate de scurtă durată.

- **Procesarea imaginilor în timp real (Real-time Processing)**

Deoarece un vehicul se deplasează cu viteză, sistemul trebuie să ia decizii instantanee. Un obiectiv major a fost optimizarea codului pentru a procesa fluxul video cadru cu cadru, cu o latență minimă, folosind biblioteci precum **OpenCV**.

- **Validarea și testarea sistemului**

Verificarea funcționalității modelului atât pe imagini statice, cât și pe fișiere video care simulează condiții reale de trafic

2 Cunoașterea și analiza setului de date

2.1 Descrierea sursei de date

Pentru antrenarea modelului, am utilizat un set de date specializat, accesat prin platforma Roboflow care conține imagini din trafic nocturn.

Imaginile din acest dataset sunt etichetate cu casete ce reprezintă fie o mașină, fie o sursa de lumina provenită de la mașini (faruri).

Datele au fost exportate în formatul compatibil YOLOv8.

Mai jos este prezentat scriptul utilizat pentru descărcarea și pregătirea automată a setului de date:

```
from roboflow import Roboflow

rf = Roboflow(api_key="hjawLDwDDSWk7QizKtUB")
project = rf.workspace("light-source-ejllw").project("ml_car_headlight")
version = project.version(1)
dataset = version.download("yolov8")
```

2.2 Mediul de implementare

Procesul de antrenare a fost realizat pe o stație de lucru echipată cu o placă video dedicată, compatibilă cu tehnologia **NVIDIA CUDA**. Utilizarea GPU-ului a fost esențială pentru paralelizarea calculelor matriciale complexe specifice rețelelor neurale convoluționale (CNN).

Resurse Software și Biblioteci: Implementarea s-a bazat pe ecosistemul limbajului **Python**, utilizând următoarele biblioteci fundamentale:

1. **PyTorch:** Cadrul de lucru de bază pentru construcția și antrenarea rețelei
2. **Ultralytics YOLO:** O bibliotecă specializată care oferă implementări optimizate ale algoritmului YOLO (*You Only Look Once*), facilitând încărcarea greutăților pre-antrenate.
3. **OpenCV:** Utilizată pentru preprocesarea imaginilor, redimensionarea și vizualizarea rezultatelor în timp real.
4. **Roboflow API:** Integrat în cod pentru gestionarea versiunilor setului de date și descărcarea automată a acestora în structura corectă de directoare.

2.3 Analiză statistică și structura datelor

Pentru a asigura o evaluare obiectivă a modelului, setul de date a fost împărțit în trei subseturi distincte

1. **Setul de Antrenare (Training Set):** Aproximativ **70%** din imagini. Acestea sunt folosite efectiv de rețea pentru a învăța trăsăturile vizuale ale vehiculelor.
2. **Setul de Validare (Validation Set):** Aproximativ **20%** din imagini. Utilizat la finalul fiecărei epoci de antrenare pentru a ajusta hiper-parametrii și a preveni fenomenul de overfitting (supra-învățare).
3. **Setul de Testare (Testing Set):** Aproximativ **10%** din imagini. Aceste date sunt complet „necunoscute” modelului în timpul antrenării și sunt folosite doar la final pentru a evalua performanța reală.



Figura 2.1 - Exemplu poză de antrenare

3 Modelarea Sistemului

3.1 Arhitectura Modelului și Configurația de Antrenare

Pentru rezolvarea problemei de detecție a obiectelor, am ales arhitectura **YOLOv8**. Spre deosebire de metodele clasice care scanează imaginea fereastră cu fereastră (sliding window), YOLO procesează întreaga imagine într-o singură trecere prin rețea.

Am utilizat versiunea **yolov8m.pt (Medium)**, care oferă cel mai bun echilibru între viteză (necesară procesării video în timp real) și acuratețe.

• Procedura de Antrenare

Procesul de învățare a fost realizat prin tehnica **Transfer Learning**. Scriptul de antrenare a fost configurat pentru a optimiza utilizarea memoriei VRAM și stabilitatea sistemului de operare Windows.

Codul folosit pentru antrenare este următorul:

```
from ultralytics import YOLO
model = YOLO('yolov8s.pt')
if __name__ == '__main__':
    print("Incepem antrenamentul")
    model.train(
        data='ML_Car_headlight-1/data.yaml',
        epochs=80,
        patience=20,
        imgsz=640,
        device=0,
        workers=0,
        batch=8
    )
    print("Antrenamentul s-a terminat cu succes.")
```

Principalii parametri definiți în cod sunt:

- **Epoci (Epochs = 80)**: Numărul de iterații complete prin setul de date.
- **Batch Size (= 8)**: Numărul de imagini procesate simultan. Această valoare a fost aleasă experimental pentru a preveni erorile de tip Out of Memory.
- **Workers (= 0)**: O setare critică pentru mediul Windows, care forțează încărcarea secvențială a datelor pentru a evita conflictele de multiprocessing.

3.2 Testarea și Validarea

După finalizarea antrenamentului, modelul rezultat (best.pt) a fost supus unei etape riguroase de testare pe imagini noi, pe care nu le-a „văzut” în timpul învățării.

Scriptul de testare încarcă greutatea salvată și rulează inferența pe directorul de test. Scopul este verificarea capacității de generalizare: modelul trebuie să recunoască faruri indiferent de unghiul mașinii sau de intensitatea luminii. În această etapă, rezultatele sunt vizualizate static, desenând cutiile de încadrare pe imaginile de test.

Rezultatele antrenamentului pot fi observate în figurile 3.1 și 3.2.



Figura 3.1



Figura 3.2

3.3 Detecție fază

• Algoritm de decizie

Logica implementată analizează fiecare cadru captat. Algoritmul funcționează pe baza unui prag de încredere setat la **0.20 (20%)**. Acest prag a fost calibrat pentru a fi suficient de sensibil la farurile îndepărtate.

Fluxul decizional este următorul:

Analiza listei de detecții: Se verifică vectorul `result.bboxes`.

Condiția de Comutare:

Dacă `len(result.bboxes) > 0` (există cel puțin un vehicul detectat), atunci păstrează **Faza Scurtă**.

Dacă `len(result.bboxes) == 0`, înseamnă că este drum liber și poate activa faza lungă.

Codul folosit este următorul:


```

results = model(frame, conf=0.20)
result = results[0]

if len(result.bboxes) > 0:
    status = "DETECTAT: Masina in fata!"
    action = ">> FAZA SCURTA <<"
    color = (0, 0, 255) # Rosu
    thickness = 2
else:
    status = "Drum Liber"
    action = ">> FAZA LUNGA <<"
    color = (0, 255, 0) # Verde
    thickness = 2

```

3.4 Implementare în timp real

După ce am văzut că modelul funcționează bine pe pozele de test, am vrut să fac un pas în plus și să văd cum s-ar comporta sistemul în realitate, pe o filmare din trafic. Am considerat că testarea pe imagini statice nu e suficientă, pentru că în trafic totul se mișcă rapid.

Când am rulat prima dată testul video, am observat o problemă practică. Deși modelul detecta mașinile, uneori le „pierdea” pentru o fracțiune de secundă (din cauza vitezei sau a luminii), după care le vedea iar. Asta făcea ca farurile simulate să se schimbe haotic de la Fază Lungă la Fază Scurtă și înapoi, de mai multe ori pe secundă.

Astfel, am aplicat un Delay Time, pentru a se asigura ca există un obstacol în față și doar după acel delay sa schimbe din fază lungă în scurtă și vice-versa.

Codul pentru această implementare este următorul:

```

if target_state != current_beam_state:

    if pending_state != target_state:
        pending_state = target_state
        last_switch_time = time.time()

    elif time.time() - last_switch_time > DELAY_SECONDS:
        current_beam_state = target_state # Abia acum schimbam oficial
        pending_state = None

else:
    pending_state = None

```

4 Concluzii

4.1 Rezultate obținute

Evaluarea s-a realizat pe un set de date de test separat. Modelul a demonstrat capacitatea de a identifica vehicule chiar și în condiții de iluminare slabă sau când doar farurile/stopurile sunt vizibile.

Graficul evolutiv (Fig. 5.1) ilustrează performanța modelului pe parcursul celor 80 de epoci de antrenare. Se pot observa următoarele tendințe:

- **Convergența Funcțiilor de Pierdere (Loss):**

Graficele `train/box_loss` și `train/cls_loss` arată o pantă descendentă constantă, indicând faptul că rețeaua a învățat progresiv să localizeze vehiculele și să le clasifice corect. Valoarea pierderii de validare (`val/box_loss`) s-a stabilizat în jurul valorii de 1.7, semn că modelul a început să generalizeze informația

- **Precizia și mAP:**

Metrica **mAP@50** (Mean Average Precision la un prag de 50%) a avut o creștere susținută, atingând un maxim de aproximativ **0.58**. Deși există fluctuații cauzate de varietatea scenariilor din setul de date (iluminare diversă, reflexii), tendința generală pozitivă confirmă capacitatea modelului de a detecta corect clasele principale (car și `lightSrc-auto`).

Matricea de confuzie (Fig. 5.2) oferă o imagine detaliată asupra erorilor de clasificare realizate de model pe setul de testare:

- **Detecții Corecte (True Positives):**

Se observă o performanță solidă pe clasele principale, modelul identificând corect **468 de instanțe** ale clasei `car` și **389** ale clasei `lightSrc-auto`. Acest lucru confirmă că, în majoritatea cazurilor, sistemul recunoaște corect vehiculele.

- **Analiza Erorilor:**

- **Confuzia cu Fundalul:**

Există un număr semnificativ de instanțe în coloana "background" (**222** pentru mașini și **352** pentru surse de lumină). Acest lucru indică o rată de "False Negatives" — adică modelul tinde să nu detecteze vehiculele aflate în condiții de vizibilitate foarte redusă, parțial ascunse sau aflate la distanță mare.

- **Confuzia Luminilor:**

Se observă **261 de cazuri** în care fundalul a fost clasificat eronat ca `lightSrc-auto`. Aceasta sugerează că modelul este uneori „păcălit” de alte surse de lumină din mediul urban, cum ar fi iluminatul stradal puternic sau reflexiile pe asfaltul umed.

Aceste rezultate sugerează că, deși modelul este funcțional și sigur (având o tendință conservatoare — preferă să rateze o detecție incertă decât să dea alarme false excesive pe mașini), performanța ar putea fi crescută pe viitor prin augmentarea setului de date cu imagini mai întunecate și prin etichetarea specifică a reflexiilor ca fiind exemple „negative”.

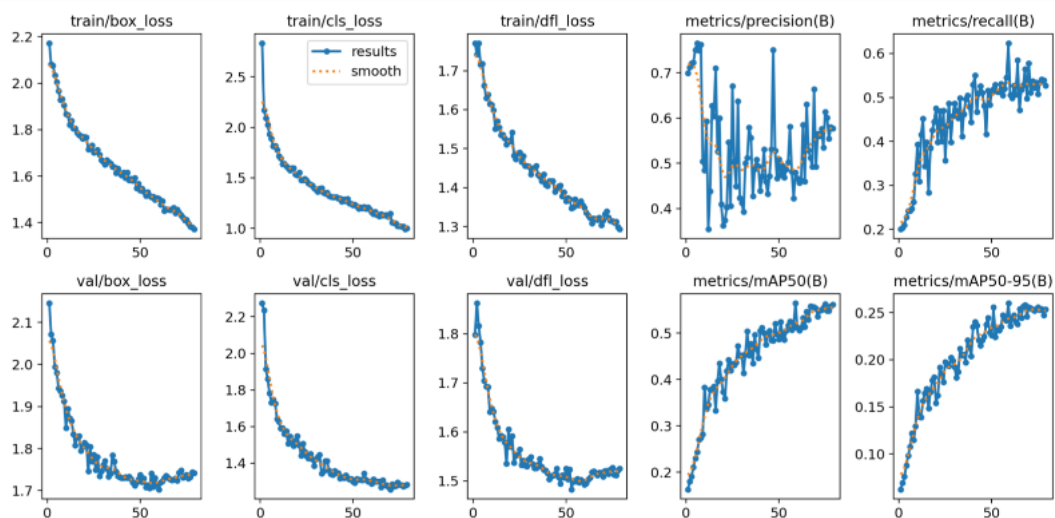


Figura 4.1 - Results

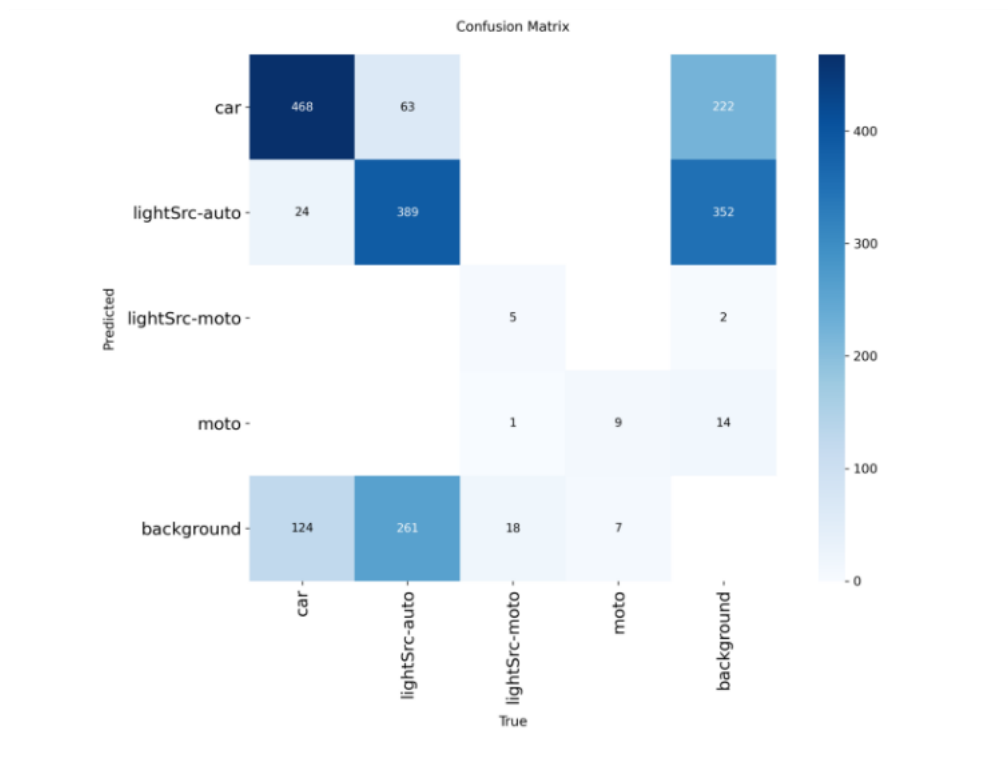


Figura 4.2 - Confusion Matrix

- Validare pe imagini statice

Figurile 4.3 și 4.4 validează funcționarea corectă a programului: în prima imagine, sistemul a detectat mașina și a specificat că trebuie folosită a **Faza Scurtă**(roșu) , iar în a doua, a detectat drum liber, și a specificat că trebuie folosită **Faza Lungă** (verde) pentru vizibilitate maximă.



Figura 4.3

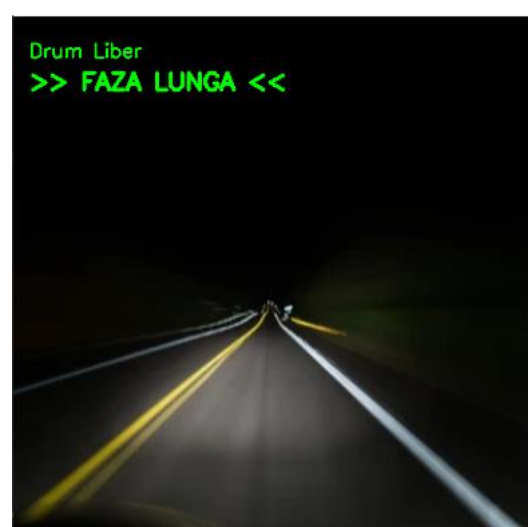


Figura 4.4

- **Integrare Video in Timp Real**

Sistemul a fost testat pe un video continuu. S-a implementat un “Dashboard” virtual care arată în timp real dacă trebuie folosită faza lungă sau faza scurtă, iar figurile 4.5 și 4.6 validează funcționarea corectă a programului.



Figura 4.5



Figura 4.6

4.2 Direcții de dezvoltare

Deși prototipul este funcțional, există câteva aspecte care pot fi îmbunătățite pentru a transforma acest proiect într-un produs comercial:

- **Diversificarea Datelor (Condiții Meteo):** Antrenarea modelului actual s-a făcut preponderent pe imagini clare. O direcție viitoare esențială este colectarea de date pe timp de ploaie, ceață sau ninsoare, situații în care reflexiile pe asfalt pot păcăli senzorii vizuali.
- **Implementarea Hardware:** Pasul următor logic este portarea codului de pe laptop pe un micro-computer dedicat, cum ar fi **Raspberry Pi** sau **NVIDIA Jetson Nano**. Acesta ar putea fi conectat fizic la un releu care să aprindă/stingă un bec LED real, nu doar să afișeze un mesaj pe ecran.
- **Optimizarea pentru Erori:** Conform matricei de confuzie, modelul are uneori tendința de a confunda luminile stradale cu farurile auto. Acest lucru poate fi corectat prin adăugarea unei clase specifice de „Lampi Stradale” în setul de date, învățând modelul să le ignore activ.

5 Bibliografie

- [1] D. M. Montserrat, Q. Lin, J. Allebach și E. J. Delp, „Training Object Detection And Recognition CNN Models Using Data Augmentation,” în *IS&T International Symposium on Electronic Imaging*, 2017.
- [2] P. Rusu Both Roxana, „Sisteme bazate pe cunoastere (SbC) – Note de curs. Indrumator de laborator”, Universitatea Tehnica din Cluj-Napoca, Facultatea de Automatica si Calculatoare, suport de curs si materiale de laborator (PDF/slideuri).
- [3] Roboflow Universe, „ML Car Headlight Dataset,” [Online]. Disponibil: https://universe.roboflow.com/light-source-ejjlw/ml_car_headlight